

Computing Platform for Autonomous Driving (CP4AD)

L7. Final Project (CAN + RTOS)

Prof. Jong-Chan Kim

Dept. Automobile and IT Convergence



국민대학교
KOOKMIN UNIVERSITY

CP4AD Final Team Project

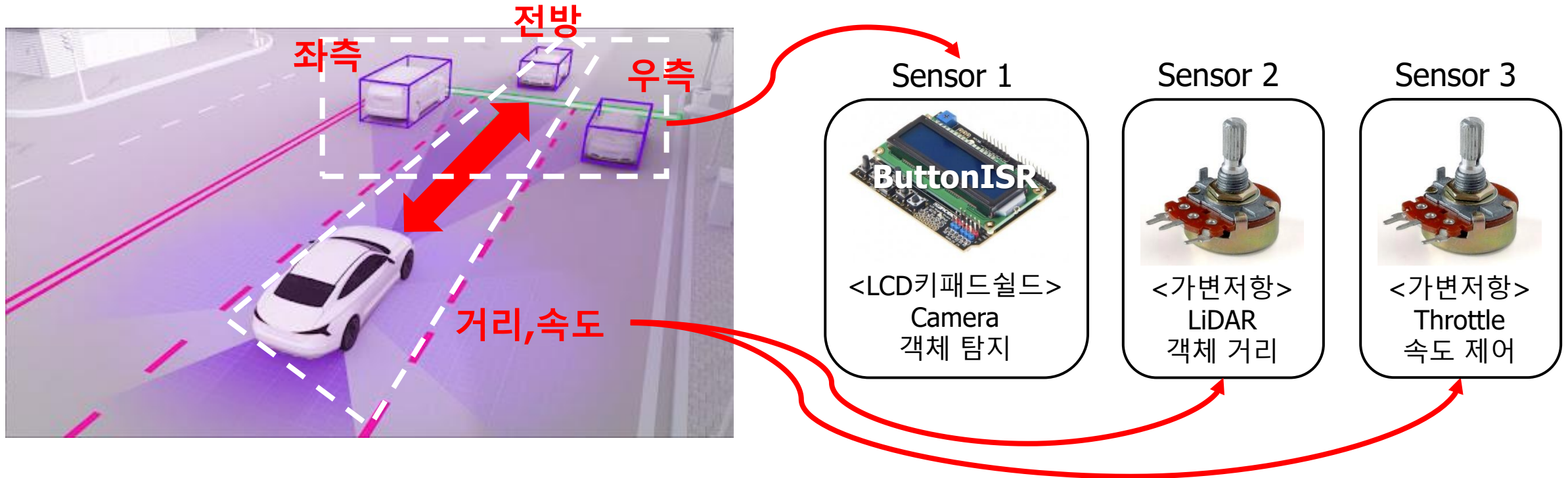
- 프로젝트 진행 후 평가 진행 → 프로젝트 점수 (20%) 반영
- 13주차 (5/27)에 OP-TEE 추가 프로젝트까지 진행 후 최종 제출 (마감 기한 : 6/3 23:59)
- 원하는 인원은 보드 대여하여 개발 가능

Logistics

- Lecture
 - Tue (14:00~17:50)
- Grading
 - Mid-term exam (20%)
 - Final exam (20%)
 - Attendance (20%)
 - Participation (20%)
 - Project (20%)
- Office Hour
 - Mon (15:00 – 16:00)
- TAs
 - 노승우 <shtmdn49@kookmin.ac.kr>
 - 안하민 <anhamin@kookmin.ac.kr>
 - 박홍균 <hongkerous2619@kookmin.ac.kr>

CP4AD Final Team Project

- RTOS, CAN, OP-TEE 기반 자율주행 장애물 감지 및 회피 시스템
- 목표:
 - ✓ RTOS에서 ISR, Task, Event, Resource 등 구조 이해
 - ✓ 실제 센서(Button/가변저항)와 연동된 CAN 통신 및 Task 스케줄링 실습
 - ✓ OP-TEE를 활용하여 보안성 강화



CP4AD Final Team Project

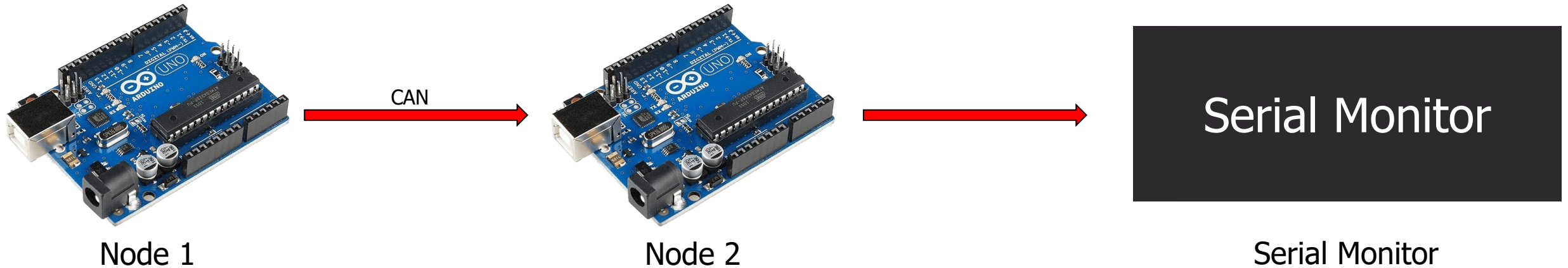
- 시스템 시나리오
 - 자율주행 차량이 주행 중
 - 전방 / 좌측 / 우측에 장애물이 감지될 때
 - 거리가 일정 값 미만이면 감속하며 회피
 - 그렇지 않으면 무시
 - 회피 동작은 반드시 자원을 점유하고 단독 실행

Task 이름	기능 설명	공유 자원
SensorTask	방향 및 거리 판단, AvoidTask 결정	없음
AvoidFrontTask	전방 회피	steer, RPM
AvoidLeftTask	좌측 회피	steer, RPM
AvoidRightTask	우측 회피	steer, RPM
CANReceiveTask	CAN 메시지 수신	없음



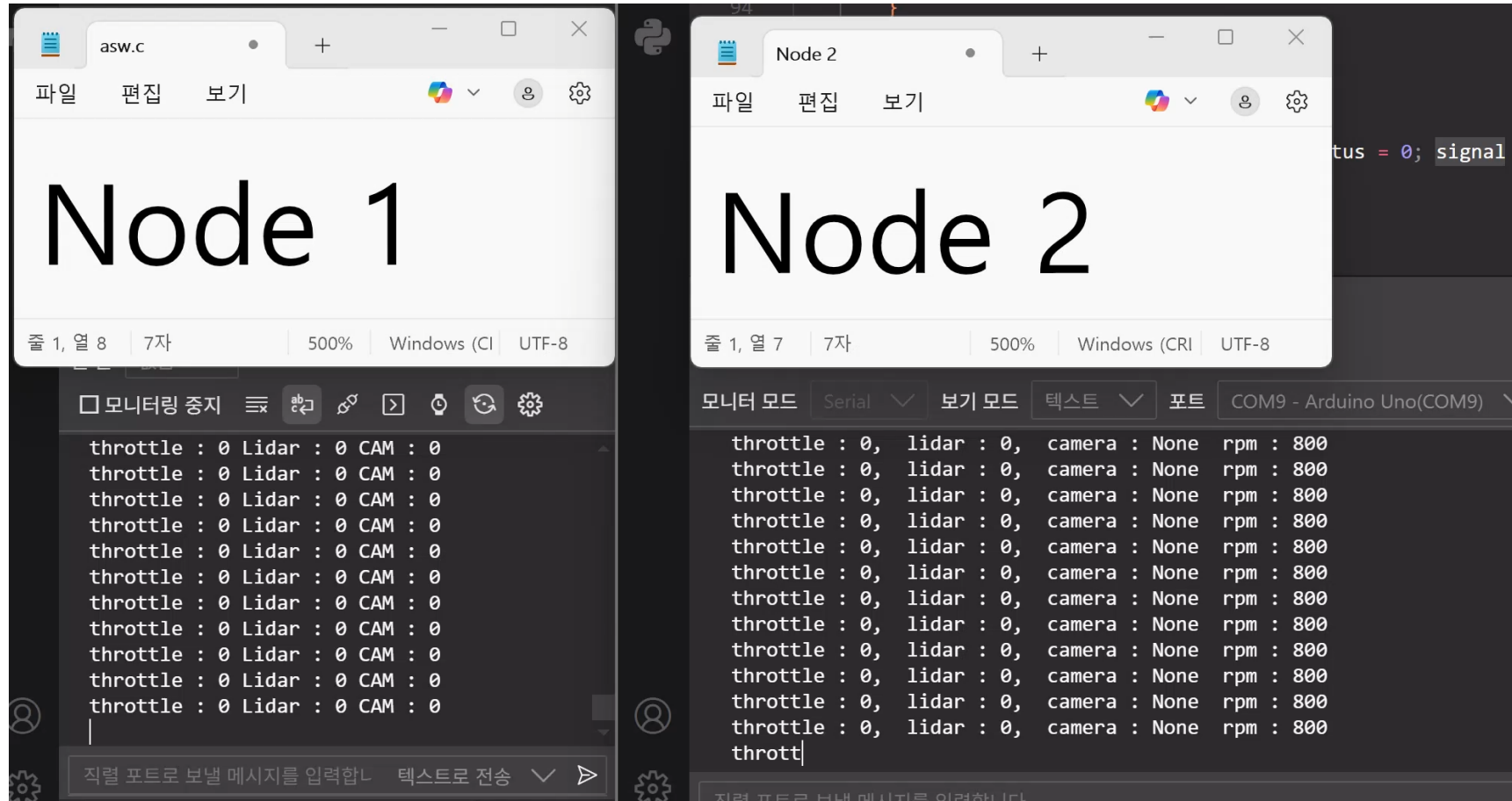
RTOS + CAN

- Node 1: 아래의 센서 값을 읽은 후 CAN을 통해 Node 2에게 전달
 - ✓ Camera : 버튼 센서
 - ✓ LiDAR : 가변저항
 - ✓ Throttle : 가변저항
- Node 2: 상황에 맞는 적절한 회피 동작 진행 및 RPM, LiDAR, Steer 등의 현재 상태 값을 Serial Monitor에 표시



Demo

- Node 1이 입력 받은 센서 데이터를 Node 2에게 CAN을 통해 전송
- RPM은 차량 모델에 Throttle 값을 입력한 후 출력된 결과



Demo

- Camera에 객체가 탐지 되었을 때 LiDAR < Threshold 라면 감속 (Threshold = 500)

Node 1

줄 1, 열 8 7자 500% Windows (CI) UTF-8

☐ 모니터링 중지

```
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 316 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lidar : 315 CAM : 0
throttle : 321 Lid
```

직렬 포트로 보낼 메시지를 입력합니다. 텍스트로 전송

Node 2

줄 1, 열 7 7자 500% Windows (CRI) UTF-8

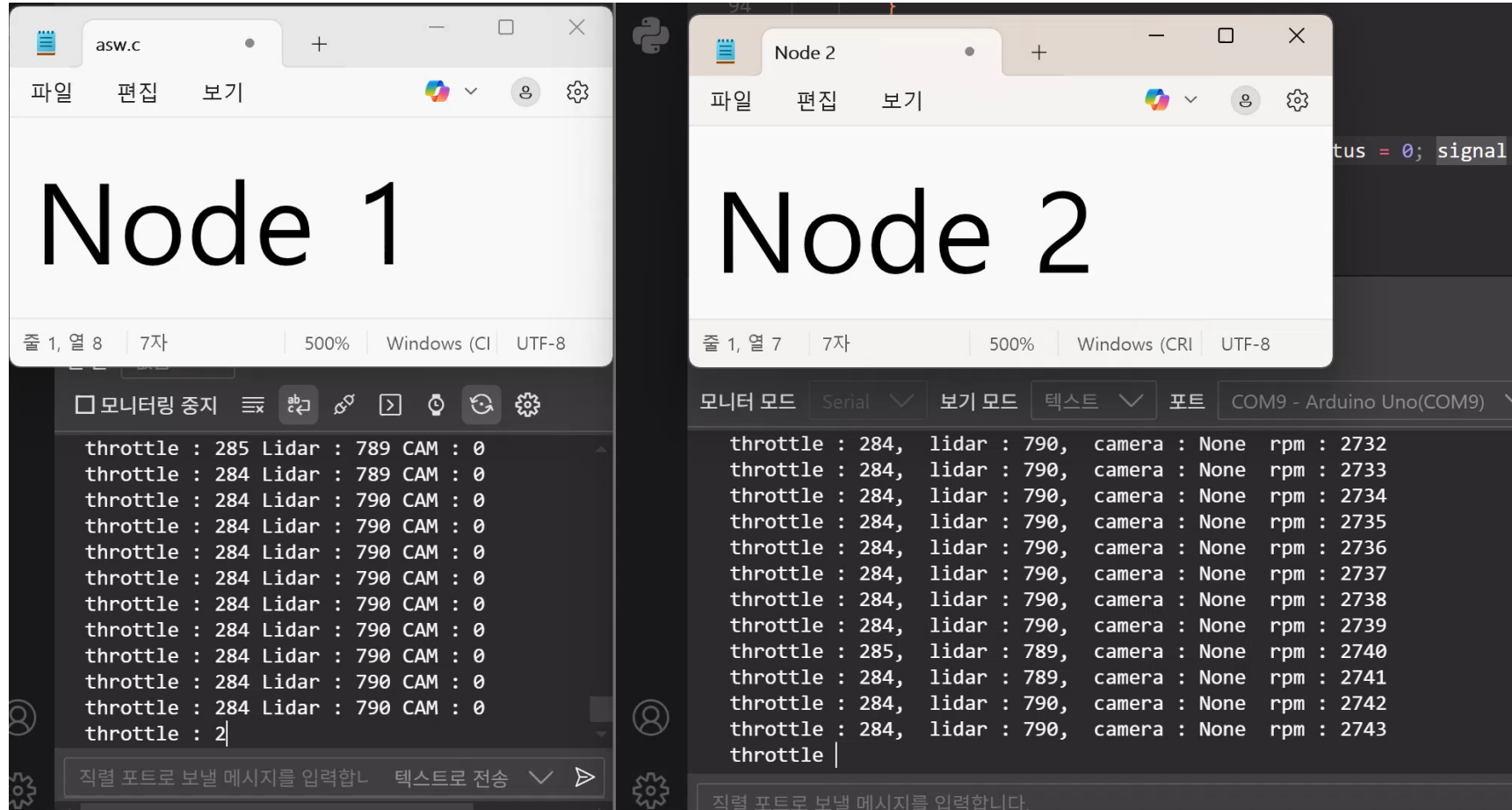
모니터 모드 Serial 보기 모드 텍스트 포트 COM9 - Arduino Uno(COM9)

```
throttle : 321, lidar : 316, camera : None rpm : 5128
throttle : 321, lidar : 315, camera : None rpm : 5130
throttle : 321, lidar : 315, camera : None rpm : 5132
throttle : 321, lidar : 315, camera : None rpm : 5134
throttle : 321, lidar : 315, camera : None rpm : 5136
throttle : 321, lidar : 315, camera : None rpm : 5138
throttle : 321, lidar : 315, camera : None rpm : 5140
throttle : 321, lidar : 315, camera : None rpm : 5142
throttle : 321, lidar : 315, camera : None rpm : 5144
throttle : 322, lidar : 315, camera : None rpm : 5146
throttle : 321, lidar : 315, camera : None rpm : 5148
throttle : 321, lidar : 315, camera : None rpm : 5150
throttle : 321, lidar : 315, camera : None
```

직렬 포트로 보낼 메시지를 입력합니다.

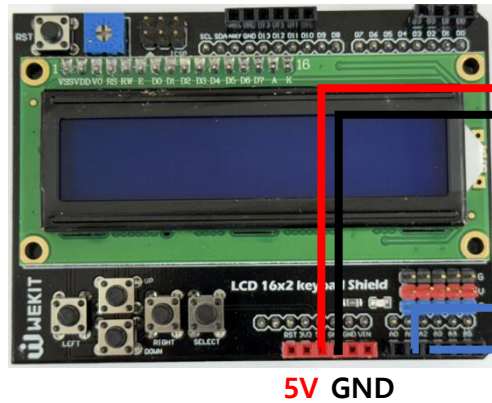
Demo

- Camera에 객체가 탐지 되었을 때 LiDAR $\geq$ Threshold 라면 감속하지 않음 (Threshold = 500)

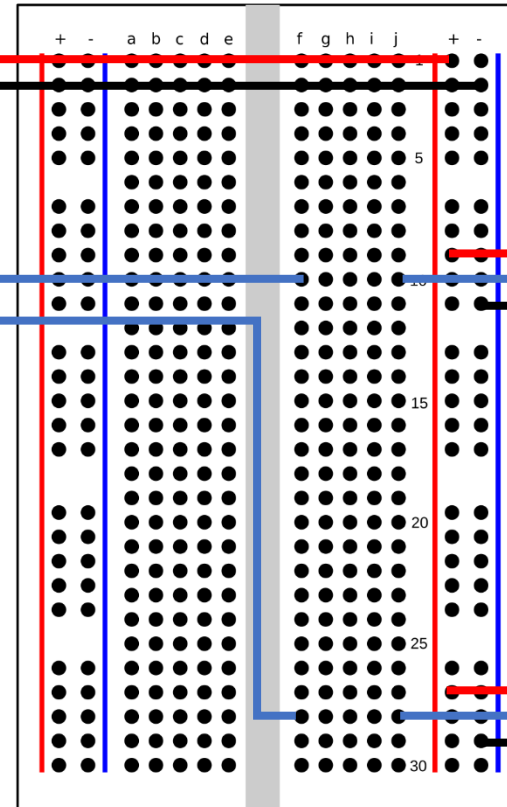
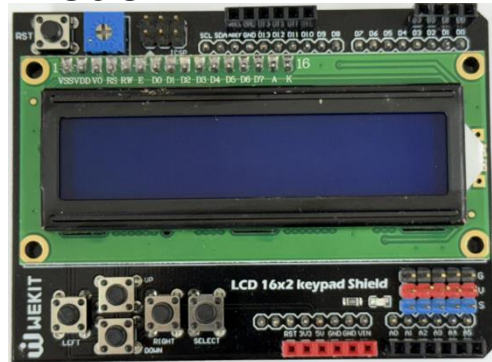


PIN MAP

Node 1



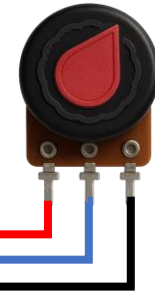
Node 2



Throttle



LiDAR



프로젝트 주의사항

- 아두이노 SRAM은 2KB로 매우 작음
- 최적화 개발이 중요함

Tech specs

Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Analog Input Pins	6
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Clock Speed	16 MHz

프로젝트 주의사항

- 초과 시 OS 멈춤, 초기화 혹은 출력 깨짐 현상 등 발생
- printfSerial 함수 사용 최소화
- printfSerial 인자는 1개만 사용 권장 및 자료형 확인
- ISR에는 많은 로직 넣지 않기 (Timer, Button, etc..)
- mdelay 사용하지 않기 (CAN 타이밍 이슈)

```

-3:
-2:
-1:
0:
1: [NORM] RPM = 1000
2: [NORM] RPM = 1000
3: [NORM] RPM = 1000
4: [NORM] RPM = 1000
5: [NORM] RPM = 1000
6: [NORM] RPM = 1000
7: [NORM] RPM = 1000
8: [NORM] RPM = 1000
9: [NORM] RPM = 1000
10: [NORM] RPM = 1000
11: [NORM] RPM = 1000
12: [NORM] RPM = 1000
13: [NORM] RPM = 1000
14: [NORM] RPM = 1000
15: [NORM] RPM = 1000
16: [NORM] RPM = 1000
17: [NORM] RPM = 1000
18: [NORM] RPM = 1000
19: [NORM] RPM = 1000
20: [NORM] RPM = 1000
Y [NORM] RPM = 1000
22: [NORM] RPM = 1000
23: [NORM] RPM = 1000
: [NORM] RPM = 1000
25: [NORM] RPM = 1000
26: [NORM] RPM = 1000

```

```

.....
...OS Starts...
.....

-3:
-2:
-1:
 0:
  1: [NORM] RPM = 1000
  2: [NORM] RPM = 1000
cc=====
  c1=====
=====Z
  c1=====
    =====Z
      =====F
        =====
          c1[c=====8=====
            =====
              c
                c=====b5=====Z
                  c1=====
                    =====
                      =====

```

프로젝트 주의사항

- bsw.cpp에 정의된 searchRemainMemory() 함수를 사용하여 남은 메모리 양 체크 가능
- 남은 메모리를 적절하게 활용하여 소스코드 작성
- CAN은 300 Byte 가량 점유

```
.....  
...OS Starts...  
.....  
  
-3: rpm : 0 rem : 919  
-2: rpm : 0 rem : 919  
-1: rpm : 0 rem : 919  
0: rpm : 0 rem : 919  
1: rpm : 1000 rem : 919  
2: rpm : 1050 rem : 919  
3: rpm : 1100 rem : 919  
4: rpm : 1150 rem : 919  
5: rpm : 1200 rem : 919  
6: rpm : 1250 rem : 919  
7: rpm : 1300 rem : 919  
8: rpm : 1350 rem : 919
```

프로젝트 주의사항

- conf.oil 에서 Event 정의 시 STACK 할당 공간 설정 가능
- 기존 실습에서는 1024로 할당하여 진행
- Event에 들어갈 코드 복잡도에 따라 적절하게 조절

Event 사용을 위한 STACK 공간을
1024 -> 128로 줄여서 할당

```
TASK SensorTask {  
    PRIORITY = 3;  
    SCHEDULE = FULL;  
    AUTOSTART = TRUE;  
    ACTIVATION = 1;  
    STACK = PRIVATE{  
        SIZE = 128;  
    };  
    EVENT = EventFront;  
    EVENT = EventLeft;  
};
```

프로젝트 주의사항

- Car_Model 사용법
- Signal = 0 → 기본 연산 진행
- Signal = 1 → base rpm 초기화

```
int16_t control_rpm(int16_t throttle_position, bool singal) {  
    if (singal == true){  
        base_rpm = 1000;  
    }  
    else{  
        int16_t gear_position = 3;  
        throttle_position = saturate(throttle_position, 0, 1023);  
        throttle_position *= 0.01;  
        gear_position = saturate(gear_position, 1, 6);  
  
        float gear_ratios[] = {3.5, 2.2, 1.5, 1.0, 0.8, 0.6};  
        float final_drive_ratio = 4.1;  
        float wheel_radius_m = 0.3;  
  
        float gear_ratio = gear_ratios[gear_position - 1] * final_drive_ratio;  
  
        float max_engine_torque = 200.0;  
        float engine_torque = (throttle_position / 100.0f) * max_engine_torque;  
    }  
}
```

TO DO

- 제출항목 : 기능들을 설명하는 PPT (형식 자유, 영상 포함), 전체 코드 압축 파일
 - ✓ PPT에는 코드설명, 기본동작 영상, 전체 시나리오 회피 동작 영상 필수 포함
- 평가항목 :
 - ✓ CAN 데이터필드 구성 및 Byte Split을 적절하게 구현했는가?
 - ✓ CAN 데이터가 딜레이 없이 원활하게 통신이 되는가?
 - ✓ RTOS 기능들을(Timer, Task, ButtonISR, Alarm, Event, Resource(or Mutex)) 모두 사용하여 구현하였는가
 - ✓ 카메라 입력(Button), LiDAR 입력(가변저항)에 따른 적절한 회피 기능이 구현되어 있는가
 - ✓ Node 2에서 현재 상태를 적절하게 시리얼 모니터에 출력하는가
 - ✓ OP-TEE (추후 안내)

Questions

